VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

COURSE TITLE

Cryptography And Network Security

COURSE CODE

BCSE309L

REVIEW - III

PROJECT TITLE

"CryptoChat v2" - From Trust Me to Prove It: A cryptographically verifiable, end-to-end encrypted chat system.

SLOT

B2+TB2

FACULTY

Dr. N G Bhuvaneswari

TEAM DETAILS

Rahul Dutta - 23BCE1250

Biswadeep Dawn - 23BCE1024

ACKNOWLEDGEMENT

The authors thank their mentor, Dr. N G Bhuvaneswari for her valuable guidance and support throughout the research and the project.

CONTENTS

Section No.	Section Title	Page No.
1	Introduction	4
2	Problem Statement	5 – 6
3	Objectives	7 – 8
4	Methodology	9 - 12
5	Algorithms	13 - 17
6	Experimental Results	18 - 22
7	Cryptanalysis	23 - 26
8	Comparative Analysis / State-of-the-Art Analysis	27 - 30
9	Conclusion	31

1. Introduction

Secure messaging has never been more important. The current messengers, including WhatsApp, Telegram, and Signal, provide various features, including anonymity and the self-destructing messages feature, but the security provided by them is based on policies rather than mathematics. CryptoChat v2 turns all assumptions of trust into cryptographic facts.

1.1 Problems with the Concept of Trust-based Security

Traditional security for message systems rests on three unverifiable trust principles:

- **Authentication of Trust:** Registration via phone number or email generates server-side metadata adequate for determining the identity of the user, despite anonymity claims.
- **Integrity of Trust:** Encryption is done within a closed-server black box, with users unable to verify if the plaintext, session key, or cipher text is stored/analyzed.
- **Deletion of Trust:** Message and session key deletion is dependent entirely on the good intentions of the service provider without any cryptographic protection.

1.2 The Paradigm Shift: From "Trust Me" to "Prove It"

The CryptoChat v2 builds up on replacing all trust assumptions with math proofs:

- **Trust Pillar 1:** The Phantom Sender: Ring signatures (the same primitive used in Monero) prove that a message is sent from one of N users, without specifying who. Chaum's Blind Signature protocol provides us with authentication tokens signed by the server while remaining blind to its eyes. Diffie-Hellman using QR or 6-digit code allows devices to pair even with no shared secrets beforehand.
- **Trust Pillar 2:** The Witness Protocol: All messages become the leaves in a Merkle tree structure, and only the 32-byte Merkle tree root is disclosed. Any message is provable for existence at some point in time T using O(log n)-sized Merkle Path, without disclosing anything about the rest of the messages, as in Bitcoin.
- **Trust Pillar 3:** Cryptographic Amnesia: Key derivation uses the Signal protocol double ratchet, and the key for a message is derived with HKDF-SHA256 function. After the ratchet step, the old key isn't deleted, but becomes mathematically impossible to retrieve. Keys generated via AES-256-GCM are automatically deleted by a background process after some timestamp. Proof of Deletion HMAC-signed.

1.3 Significance and Scope

CryptoChat version 2 is a fully launched application consisting of a backend built with Railway (Flask 3.0 + Flask-SocketIO) and a frontend using Vercel (React 19). All cryptography operations are executed on the client side through the use of the native WebCrypto API, and hence, the browser is considered as an independent cryptographic authority. This project incorporates nine cryptography primitives, namely, Ring Signatures, Blind Signatures, Zero Knowledge Proofs, Diffie-Hellman key exchange, Double Ratchet, Merkle Trees, Commitment schemes, Time-Lock ciphers, and Proof of Deletion.

2. Problem Statement

2.1 Formulation of the Problem

The problem considered in this project can be formulated as follows:
Current methods of encrypted communications do not provide mathematical proof of security, as all proofs of security are policy-based. Users cannot individually verify that their identities have not been matched, their messages are not logged, and the keys have been properly destroyed. Users will be exposed to attacks originating from compromised servers, malicious insiders, subpoenas, and traffic analysis.

2.2 Identified Gaps in Security of Existing Platforms

Today's messaging systems fall short in at least five important categories:

- **Identity Linking:** registering using phone or email associate user identity with the messages.
- **Poor Encryption:** there is no way for the user to ensure that the plaintext, session keys, and ciphertexts are not stored on the server.
- **Message Deletion Not Verifiable:** it is simply deleting from a database on the server side.
- **Compromised Forward Security:** less frequent ratchet leads to decryption of all previous conversations after key compromise.
- **Replay Attacks:** without server-side nonce tracking, intercepted messages can be re-transmitted and processed multiple times.

2.3 Limitations of Existing Approaches

The table below summarises the security properties of three widely-used messaging platforms against the five identified gaps:

Security Property	WhatsApp	Telegram	Signal	CryptoChat v2
Provable Sender Anonymity	no	no	no	yes
Verifiable Content Non-Storage	no	no	no	yes
Cryptographic Deletion Proof	no	no	no	yes
Per-Message Forward Secrecy	Partial	no	yes	yes
Anti-Replay Nonce Registry	no	no	Partial	yes
Independent Auditability	no	no	Partial	yes
Server-Trust Independence	no	no	no	yes

Although WhatsApp utilizes the Signal Protocol, it does not have any encryption at all due to being owned by Meta and having metadata collection practices confirmed. Regular Telegram chats have no end-to-end encryption, and even secret Telegram chats depend on Telegram's servers to manage keys. The best application among these three is Signal, but it is not secure from metadata collection.

2.4 Core Research Problem

The core research problem as per the discussion made above can be defined as follows:
How can an end-to-end encryption message service be designed that guarantees and proves sender anonymity, message nonstorage, forward security, cryptographic message deletion, and prevention from replay attacks, independent from any cooperation of the server?

2.5 Threat Model

The following types of adversaries must not be able to breach the security of the system:

- **Honest-But-Curious Server:** Operates per the protocol without malicious intent but tries to gather as much info as possible. The design makes sure that the server gains no more knowledge than mathematically necessary.
- **Malicious Server:** Tries to fake messages, impersonate users, or correlate identities. Ring Signatures and Safety Numbers prevent this because forging is as difficult as breaking RSA-2048 while correlation within the ring is mathematically impossible.
- **Network-Level Adversary:** Can control network traffic and tries MITM, modifying, or replaying messages. Nonce accounting prevents replay attacks; authenticated encryption prevents modifications; Double Ratchet prevents past message recovery.
- **Future Compromise Adversary:** Gains access to current secret key material, say via seizing devices. Forward secrecy provided by Double Ratchet makes sure all past secrets will be erased and irretrievable.

3. Objectives

The goals of the v2 version of CryptoChat are specified in terms of three distinct categories: cryptographic goals that aim to solve the discovered security problems, system goals that dictate the design and implementation of the system itself, and evaluation goals that define metrics to determine whether the system meets its requirements.

3.1 Cryptographic Goals

- **Provable Sender Anonymity: The** Ring Signature must certify that the message was created by one of N registered users along with Chaum's Blind Signature for token authentication without revealing its contents.
- **Provable Message Presence:** Merkle Tree must be implemented with messages used as leaves and 32-byte root hash allowing O (log n) membership proofs without leaking any other messages due to commit-and-reveal ability.
- **Forward Security per Message:** The HKDF-SHA256 key must be used to encrypt every message, which is generated per ratchet state transition that makes all previous states unrecoverable for both Python and JavaScript implementations.
- **Provable Key Destruction:** Upon expiration, the key wiping proof signed with HMAC must show that the key has been wiped to zeros based solely on attestation key of the server, while AES-256-GCM key zeroization happens once every 60 seconds via scheduler.
- **Device Pairing:** Two devices should generate a session key using Diffie-Hellman (RFC 3526 2048-bit MODP Group 14) using PBKDF2-SHA256 (100,000 iterations), which is available either by QR code or by Safety number.
- **Nonce Anti-Replay Protection:** All nonces received by the server must be stored across restarts.

3.2 Objectives of the System Level

- **No Server-Side Content Storage:** messages cannot be saved as plain text, but only hash values, Merkle state, and nonces can be stored.
- **Real-Time Encryption:** devices have to be able to communicate through WebSocket without significant delays.
- **Full Implementation in Production Environment:** back-end implementation on Railway using Gunicorn/Eventlet, front-end as React SPA on Vercel, CORS and WebSocket proxy support, WebCrypto headers, and persistence through file-based and GitHub API implementations.
- **Security Events Tracking:** logging of authentication errors, replay attacks, brute force (≥5 incorrect logins per IP address during 5 minutes), detection of MITM attack, and expired keys with timestamps, severity tags, REST API accessibility, and live streaming in an admin panel.
- **Only Client-Side Cryptography:** all cryptographic functions can utilize only the WebCrypto API provided by the client browser.

3.3 Objectives for Evaluation

- **Zero Successful Attempts:** zero successful attempts for replay attacks, brute-force attacks, MITM attacks, and any other unauthorized access should be registered by the system during penetration testing.
- **Independent Merkle Auditability:** the leaf hash and Merkle path of any particular message should be enough for calculating the root hash by means of REST API independently of any other messages.
- **Forward Secrecy Checking:** the Double Ratchet algorithm must generate one unique key per message without any advantage if K_i is known for generating K_{i-1}, checked on both Python and JavaScript platforms.
- **Cryptographic Proof of Deletion:** once the key expires, Proof of Deletion message becomes checkable with the attestation public key only and all messages encrypted with it can no longer be decrypted.

3.4 Summary of Objectives

The following table summarizes the mapping between the objectives and their respective cryptographic techniques, along with the sections in the report that provide clear descriptions of how they are implemented.

Objective	Cryptographic Mechanism	Detailed In
Provable Sender Anonymity	Ring Signatures, Blind Signatures	Section 4.4, 4.5
Verifiable Message Existence	Merkle Trees, Commitment Schemes	Section 4.6, 4.7
Per-Message Forward Secrecy	Double Ratchet, HKDF-SHA256	Section 4.8
Cryptographic Key Deletion	Time-Lock Cipher, Proof of Deletion	Section 4.9, 4.10
Secure Anonymous Pairing	Diffie-Hellman, Safety Numbers	Section 4.3
Anti-Replay Protection	Nonce Registry	Section 4.11
Zero Content Storage	Architecture Design	Section 4.2
Real-Time Communication	WebSocket, Flask-SocketIO	Section 4.2
Production Deployment	Railway, Vercel, GitHub Storage	Section 4.2
Security Monitoring	SecurityMonitor, APScheduler	Section 4.12

4. Methodology

The methodology of CryptoChat v2 follows a **cryptography-first design**, where each system component is built to eliminate specific trust assumptions. Instead of relying on policies, the system ensures that security properties such as anonymity, integrity, and deletion are enforced through verifiable cryptographic mechanisms. The approach is structured across multiple dimensions, including cryptographic design, system architecture, implementation, and evaluation. Each layer contributes independently to building a secure, verifiable, and trust-independent communication system.

4.1 Cryptographic Design Methodology

4.1.1 Criteria for Primitive Selection

The selection of cryptographic primitives is based on provable security, composability, and independent verifiability. Each primitive is grounded in well-established mathematical hardness assumptions such as RSA and discrete logarithms, ensuring resistance to known attacks.

Additionally, primitives are designed to interoperate securely without introducing vulnerabilities. Outputs from one mechanism can be safely used by another, while each component remains independently verifiable without relying on server-side trust.

4.1.2 Cryptographic Architecture's Three Pillars

- **Pillar 1 — The Phantom Sender:** addresses identity link ability using ring signatures, blind signatures, and secure device pairing. It ensures that the sender's identity remains computationally hidden, making it infeasible for the server to determine the actual origin of any message.
- **Pillar 2 — The Witness Protocol:** ensures verifiable message existence through Merkle trees and commitment schemes. It allows independent verification of message integrity and timestamp without exposing message content.
- **Pillar 3 — Cryptographic Amnesia:** enforces secure deletion and forward secrecy using the Double Ratchet algorithm, time-lock encryption, and proof of deletion. Once keys are expired or discarded, recovery becomes computationally infeasible, ensuring irreversible data loss.

4.1.3 Two-Layer Implementation Approach

The cryptographic logic is implemented consistently across both backend and frontend environments. Algorithms such as Double Ratchet are executed using identical parameters in Python and JavaScript to ensure synchronization and correctness.

All encryption and decryption operations occur at the endpoints, while the server functions only as a relay and verification layer. This ensures that sensitive data is never exposed outside user-controlled devices.

4.2 System Architecture Methodology

4.2.1 Separation of Concerns

The system is divided into three layers: cryptographic, communication, and storage. The cryptographic layer handles all encryption and proof generation, the communication layer manages real-time messaging, and the storage layer stores only cryptographic metadata.

This separation improves modularity and security, ensuring that plaintext data is never processed or stored outside the endpoints.

4.2.2 Backend Framework Selection

Flask is used as the backend framework due to its lightweight and modular structure, allowing seamless integration of cryptographic modules. Flask-SocketIO enables real-time communication, while asynchronous processing ensures efficient handling of concurrent connections.
The backend acts as a coordination and verification layer without accessing or storing sensitive message content.

4.2.3 Frontend Framework Selection

The frontend is implemented using React and Vite, enabling a responsive and component-based interface. Each component corresponds to a specific functionality such as pairing or encrypted communication.

All cryptographic operations are executed using the WebCrypto API, ensuring that encryption is performed entirely on the client side.

4.2.4 Persistent Storage Architecture

A dual-layer storage approach is used to handle server restarts and persistence. Data is first written locally and then synchronized with a remote repository using API-based storage mechanisms.

Only cryptographic metadata such as hashes, proofs, and nonces are stored, ensuring that sensitive data remains protected even in case of storage compromise.

4.3 Implementation Methodology

4.3.1 Module-by-Module Development Approach

A bottom-up development approach is followed where cryptographic primitives are implemented and tested independently before integration. These modules are later combined through APIs and WebSocket handlers to form the complete system.

This ensures correctness, modularity, and reduces the risk of introducing system-wide vulnerabilities.

4.3.2 Security-First Development Principles

All cryptographic operations rely on standardized libraries such as PyCryptodome and WebCrypto, avoiding insecure custom implementations. Secure random number generation is enforced using system-level entropy sources.

Error handling is designed to prevent leakage of sensitive information, ensuring that failure cases do not expose exploitable patterns.

4.3.3 Real-Time Security Monitoring Integration

A centralized monitoring system tracks all security-related events such as replay attacks, brute-force attempts, and unauthorized access. Events are logged with structured metadata and severity classification.

A scheduler operates independently to enforce key expiration, ensuring consistent application of security policies without user intervention.

4.4 Security Evaluation Approach

4.4.1 Automated Penetration Testing

The system is tested against multiple attack scenarios including replay attacks, brute-force attempts, man-in-the-middle attacks, and unauthorized access. These tests simulate real-world adversarial behavior.

Results confirm that attacks are detected and mitigated without compromising system integrity.

4.4.2 Cryptographic Validation

Cross-platform validation ensures consistency between backend and frontend implementations. Encrypted messages are verified across environments, and Merkle proofs are independently recomputed to confirm correctness.

The proof-of-deletion mechanisms are also validated to ensure that expired keys cannot be reused.

4.4.3 End-to-End System Validation

Complete user workflows including pairing, messaging, and deletion are tested to ensure correct functionality. The system verifies that no plaintext is stored and that all security events are properly logged.

This ensures that both functional and security requirements are consistently maintained.

4.5 Ring Signatures — Provable Sender Anonymity

Ring signatures enable anonymous authentication by allowing a message to be signed by one member of a group without revealing which member performed the signing. Verification confirms that the signer belongs to the group without identifying them.

This ensures strong anonymity and prevents identity tracing, as the probability of identifying the actual sender is computationally infeasible.

4.6 Blind Signatures — Anonymous Token Issuance

Blind signatures allow the server to sign authentication tokens without seeing their actual content. The client blinds the token before submission and later unblinds the signed result.

This ensures that authentication is valid while preventing the server from linking tokens to specific users, thereby maintaining anonymity.

4.7 Merkle Trees — Verifiable Message Existence

Messages are represented as hash values and stored as leaves in a Merkle tree. The root hash acts as a compact representation of all messages and enables efficient verification.

Using Merkle proofs, any message can be verified independently without revealing other messages, ensuring both integrity and privacy.

4.8 Double Ratchet Algorithm — Per-Message Forward Secrecy

The Double Ratchet algorithm generates a new encryption key for every message using a one-way key derivation process. Each key is used only once and then discarded.

This ensures forward secrecy, meaning that compromise of a current key does not affect past communications.

4.9 Time-Lock Cipher and Proof of Deletion — Cryptographic Amnesia

Time-lock encryption associates keys with expiry timestamps, after which they are permanently deleted. A scheduler enforces this deletion automatically.

Proof of deletion provides verifiable evidence that a key has been destroyed, ensuring that deletion is not dependent on trust.

4.10 Nonce Registry — Anti-Replay Protection

Each message includes a unique nonce to prevent replay attacks. The server maintains a persistent registry of used nonces and rejects duplicate entries.

This ensures that previously captured messages cannot be reused, even across server restarts.

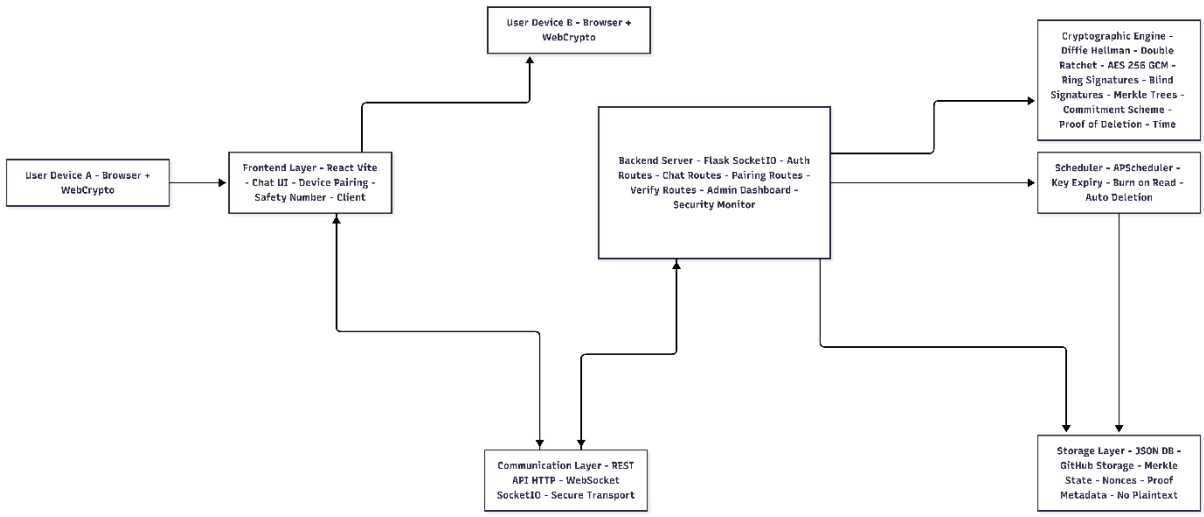
4.11 Security Monitor and Expiry Scheduler

The Security Monitor records all system events with timestamps, severity levels, and contextual details. It also detects suspicious patterns such as repeated failed attempts or replay attacks.

The expiry scheduler periodically removes expired keys, ensuring that data lifecycle policies are enforced automatically and consistently.

4.12 Block and Architecture Diagram

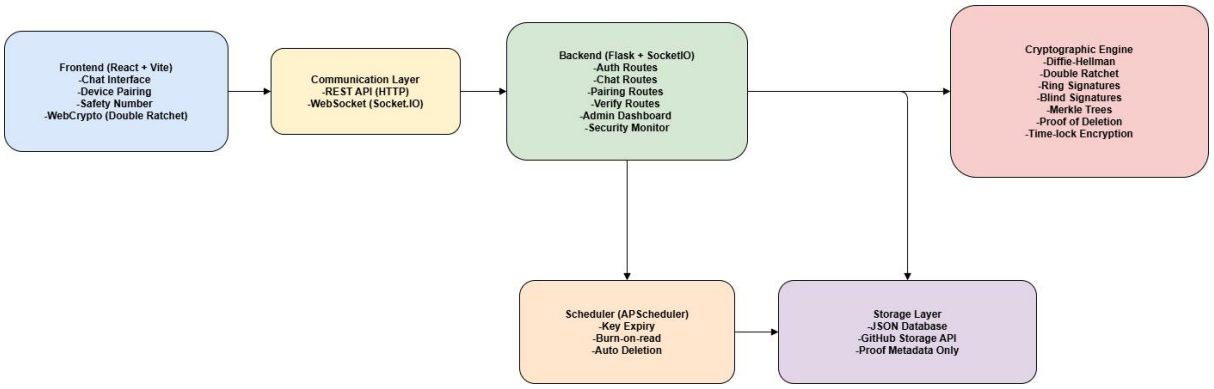
4.12.1 Block Diagram



Explanation of Block Diagram

This can be seen from the block diagram above showing the flow of information in CryptoChat v2. In the frontend, the users interact and their information is encrypted using Double Ratchet. The communication layer includes REST and WebSocket; therefore, the information goes into the backend, where encrypted information and cryptographic proofing take place, but there is never any contact with plain text information. The storage layer includes the cryptographic proof hash only.

4.12.2 Architecture Diagram



Explanation of the Architecture Diagram

The architecture is made up of five layers. The first one is the Frontend Layer (React/Vite), where chat, device pairing, and safety number verification happen, with encryption taking place on the client side through WebCrypto. The second layer, referred to as the Communication Layer, is in charge of routing REST API calls (auth, pairing, and verification) and the real-time message passing through Socket.IO WebSockets. The Backend Layer (Flask/Flask-SocketIO) controls routes and security monitoring without plaintext. The Cryptographic Engine contains all nine primitives.

5. Algorithms

The security of CryptoChat v2 is built on a combination of proven cryptographic algorithms. Unlike traditional systems that rely on trust-based policies, it uses mathematical techniques to ensure privacy, integrity, anonymity, and accountability. Each algorithm contributes a specific security function, and together they form a defense-in-depth system that eliminates the need for trusting any central entity.

5.1 Diffie-Hellman Key Exchange Algorithm

5.1.1 Overview

The Diffie-Hellman (DH) algorithm is used to securely establish a shared secret between two devices over an insecure communication channel. It forms the foundational step in initiating encrypted communication in CryptoChat v2.

5.1.2 Algorithm Description

Let:

- p= large prime number (2048-bit MODP group)
- g= generator

Steps:

1. Device A selects private key a, computes public key
$$A = g^a \bmod p$$
2. Device B selects private key b, computes public key
$$B = g^b \bmod p$$
3. Shared secret computation:
    - Device A computes $S = B^a \bmod p$
    - Device B computes $S = A^b \bmod p$

Both devices derive the same shared secret S.

5.1.3 Implementation in CryptoChat v2

- During the device pairing stage (QR/6 digit code)
- The shared secret is derived via PBKDF2-SHA256
- The result is a 256-bit encryption session key used to encrypt further data

5.1.4 Security Significance

- Prevents key exchange from being intercepted
- Provides a secure communication foundation without any prior trust

5.2 Double Ratchet Algorithm

5.2.1 Overview

Double Ratchet provides forward secrecy and post-compromise security by ensuring that each message uses a different encryption key.

5.2.2 Algorithm Description

Let:

- Initial shared secret = K_0

For each message:

1. Apply HKDF:
    - $(K_{msg}, K_{next}) = HKDF(K_{current})$
2. Encrypt message:

- $C = AES_GCM(K_{msg}, plaintext)$

3. Update chain:
   - $K_{current} = K_{next}$
4. Discard K_{msg}

5.2.3 Pseudocode

```
chain_key = initial_shared_secret

for each message:
    message_key, chain_key = HKDF(chain_key)
    ciphertext = AES_GCM_Encrypt(message_key, plaintext)
```

5.2.4 Implementation in CryptoChat v2

- Implementation in:
  - Backend (Python)
  - Frontend (WebCrypto API)
- Provides synchronization during encryption/decryption process on various devices.

5.2.5 Security Significance

- Each message has its own key
- Old keys cannot be found out if they have been thrown out
- Secures old and new messages from being decrypted even after they have been compromised.

5.3 AES-256-GCM Encryption Algorithm

5.3.1 Overview

AES-256-GCM is used for encrypting all messages, providing both confidentiality and integrity.

5.3.2 Algorithm Description

Input:

- Plaintext
- 256-bit key
- Initialization Vector (IV)

Output:

- Ciphertext
- Authentication Tag

5.3.3 Implementation in CryptoChat v2

- Applied after deriving the key through the Double Ratchet algorithm
- Every message encrypted with:
  - A unique key
  - Unique IV

5.3.4 Security Significance

- Prevents unauthorized decryption
- Detecting any tampering with messages
- Protection against replay attacks

5.4 Merkle Tree Algorithm

5.4.1 Overview

The Merkle Trees are deployed to provide proof of existence of messages without disclosing the message details.

5.4.2 Algorithm Description

1. Each message is hashed:

$$H_i = SHA256(message_i)$$

2. Pairwise hashing:

$$H_{parent} = SHA256(H_{left} + H_{right})$$

3. Continue until root hash is obtained

5.4.3 Pseudocode

```
function build_merkle_tree(leaves):
    while len(leaves) > 1:
        new_level = []
        for i in range(0, len(leaves), 2):
            combined = hash(leaves[i] + leaves[i+1])
            new_level.append(combined)
        leaves = new_level
    return leaves[0]
```

5.4.4 Implementation in CryptoChat v2

- Each message ↔ Leaf node
- Dynamic tree reconstruction
- Store root hash and distribute
- Create proof path for validation

5.4.5 Security Significance

- Maintains data integrity
- Facilitates proof of existence
- Identifies any changes made to stored data

5.5 Ring Signature Algorithm

5.5.1 Overview

Ring signatures allow for anonymous authentication of messages as coming from a particular group.

5.5.2 Algorithm Description

Given:

$$\{PK_1, PK_2, \ldots, PK_n\}$$

- A signer uses their private key to generate a signature
- Verification confirms:
    - Signature is valid within the group
    - Sender identity is hidden

5.5.3 Implementation in CryptoChat v2

- Implemented when sending messages
- The sender's identity is kept secret among various senders

5.5.4 Security Significance

- Anonymity is guaranteed
- No way of determining the identity of the sender
- Unlink ability is assured

5.6 Blind Signature Algorithm

5.6.1 Overview

Blind signatures allow a server to sign a message without knowing its content, ensuring anonymous authentication.

5.6.2 Algorithm Description

1. Blinding:

$$m' = m \cdot r^e \bmod n$$

2. Signing:

$$s' = (m')^d \bmod n$$

3. Unblinding:

$$s = s' \cdot r^{-1} \bmod n$$

5.6.3 Implementation in CryptoChat v2

- For generating authentication tokens
- The server cannot relate the token to the user

5.6.4 Security Significance

- Provides anonymity
- Prevents traceability
- Maintains confidentiality of user identity

5.7 Commitment Scheme Algorithm

5.7.1 Overview

Commitment schemes guarantee that once a message is committed, its contents cannot be changed.

5.7.2 Algorithm Description

1. Commitment:

$$C = SHA256(message + salt)$$

2. Reveal:
   - Provide message and salt
   - Verify hash matches

5.7.3 Implementation in CryptoChat v2

- Prior to transmission of the message
- Ensures integrity of the message over time

5.7.4 Security Significance

- Prevents any alteration
- Guarantees non-repudiation

5.8 Proof of Deletion Algorithm

5.8.1 Overview
The proof of deletion algorithm guarantees that deletion of cryptographic keys can be proved.

5.8.2 Algorithm Description
1. Commit to key:

$$C = H(key)$$

2. On deletion:
    - Overwrite key
    - Generate HMAC-based proof
3. Publish proof

5.8.3 Implementation in CryptoChat v2
- Initiated when:
    - Expiration of key
    - Burn-on-read mode

5.8.4 Security Significance
- Provides secure deletion
- Eliminates need for trust

5.9 Time-Lock Encryption Algorithm
5.9.1 Overview
Time-lock encryption guarantees that messages cannot be accessed after a predetermined period of time.

5.9.2 Algorithm Description
- Encryption key associated with expiry timestamp
- Scheduler checks periodically:
    - If expired → key is nullified

5.9.3 Implementation in CryptoChat v2
- APScheduler executes after every 60 seconds
- Expired keys are overwritten

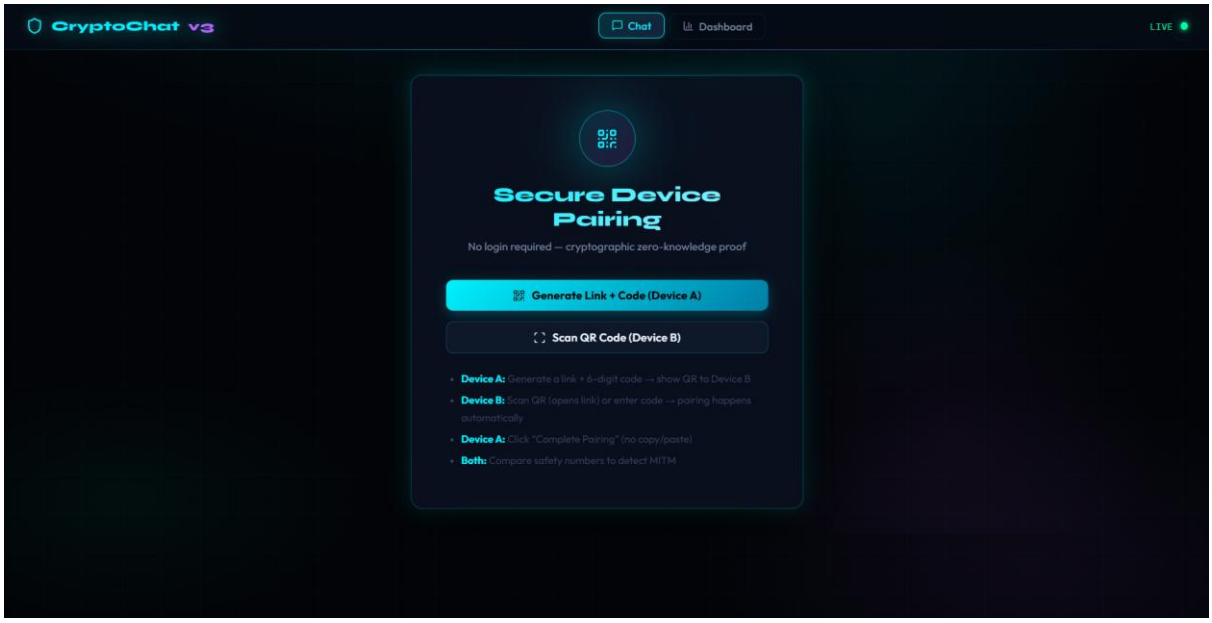
5.9.4 Security Significance
- Ensures automatic expiry of data
- Cannot be decrypted again

5.10 Summary of Algorithms
The algorithms used in CryptoChat v2 collectively provide a comprehensive security framework:
- Diffie-Hellman → Secure key exchange
- Double Ratchet → Forward secrecy
- AES-GCM → Confidentiality and integrity
- Merkle Tree → Verifiable data existence
- Ring Signatures → Anonymous communication
- Blind Signatures → Untraceable authentication
- Commitment Scheme → Data integrity
- Proof of Deletion → Verifiable erasure
- Time-Lock Encryption → Controlled data lifetime

6. Experimental Results

Experimental assessment of CryptoChat v2 shows the success of the design and development of a secure and private communication system based on cryptographic techniques. The testing was conducted for several modules such as device pairing, messaging in an encrypted form, data removal, and detecting attacks. Below are the findings of the experiment.

6.1 Front Dashboard

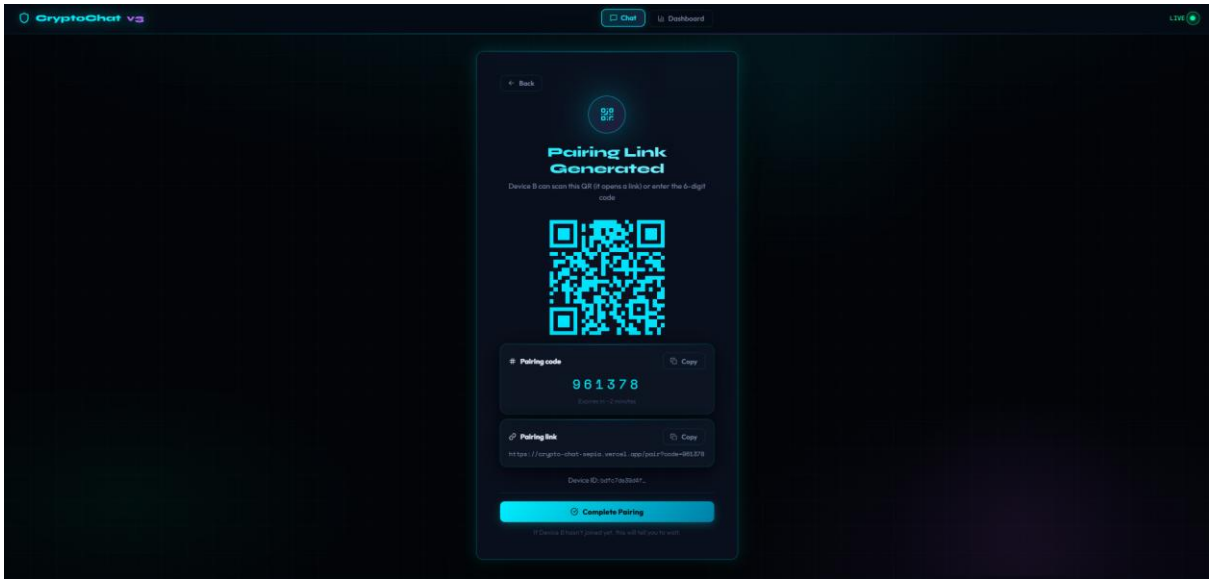


Explanation

The above interface is the first dashboard of the CryptoChat v2 application. The application offers various methods that allow users to initiate the process of device pairing in a secure environment without having to authenticate themselves using any usernames or passwords.

The existence of a clean and simple interface means that security is maintained alongside ensuring usability. The lack of traditional authentication measures (usernames and passwords) is a pointer to the design philosophy of the application.

6.2 Pairing Dashboard

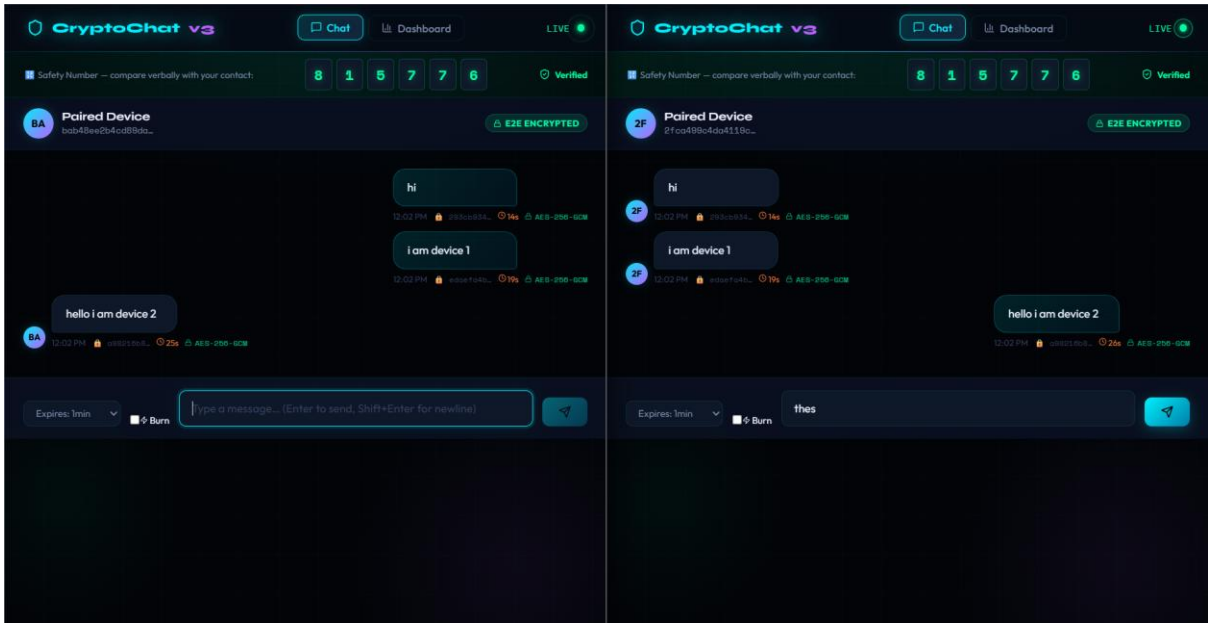


Explanation

The following figure is an example of device pairing where Device A creates a link and pairing code, and Device B either scans or enters that code to establish pairing. Both devices compute a shared key using Diffie-Hellman Key Exchange protocol. Pairing guarantees the following aspects:

- Establishment of a secure key without any exchange of keys
- Immunity against man-in-the-middle attacks
- Establishment of the same session key in both devices

6.3 Communication Between Users

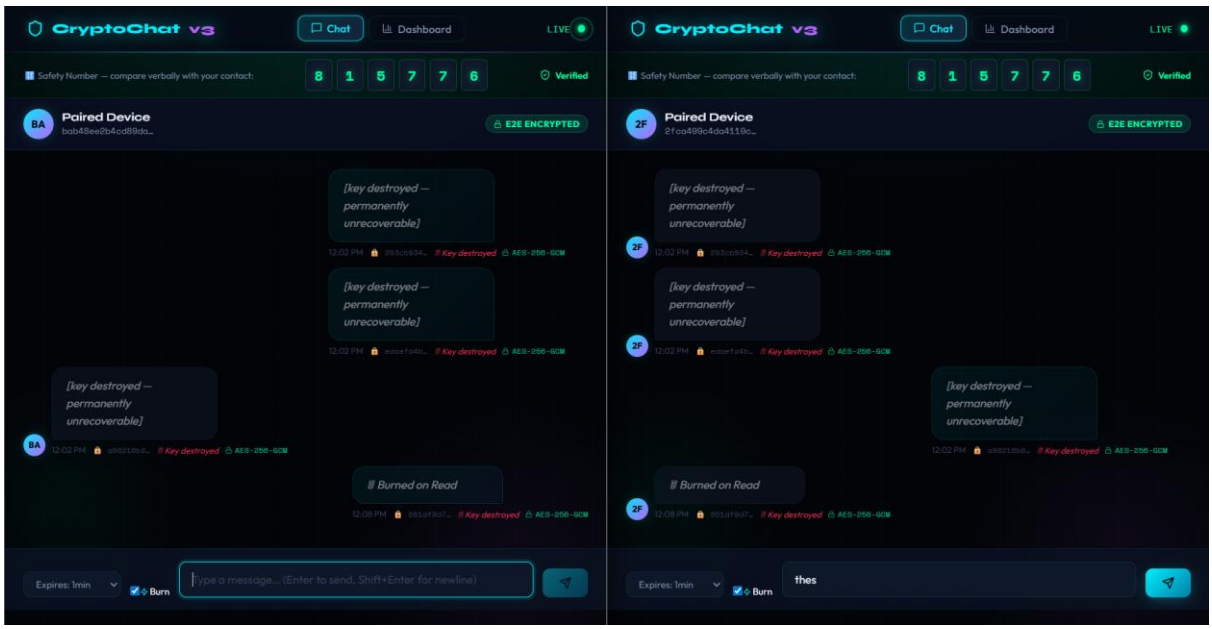


Explanation

The screen capture shown above depicts live interaction between the two connected devices. Encryption of the messages occurs via AES-256-GCM and Double Ratchet. Every single message corresponds to an encryption key, thus providing forward secrecy.

Also, the safety number visible at the top helps assure that the two users are indeed talking through a secure channel.

6.4 Message Deletion and Burn Feature



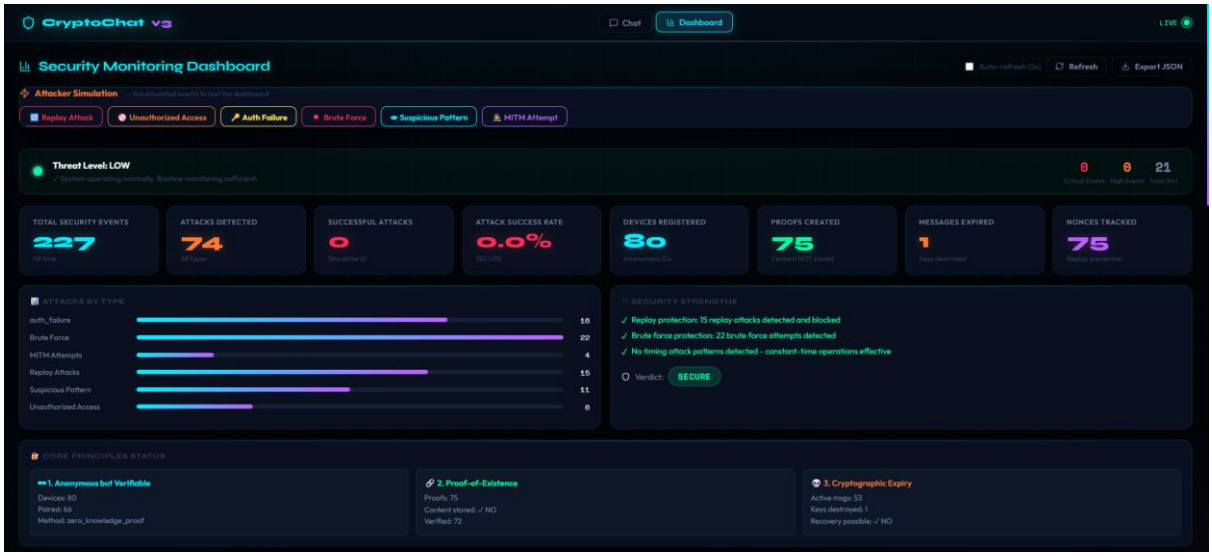
Explanation

This figure demonstrates the burn-on-read and time-based expiration of messages. Messages will be deleted when they have been read by the receiver or after an elapsed time frame.
Once expired:

- Keys for encryption are destroyed
- The messages are mathematically impossible to recover
- Deletion proof is provided

This guarantees secure data protection and prevents any unauthorized access.

6.5 Overall Security Dashboard

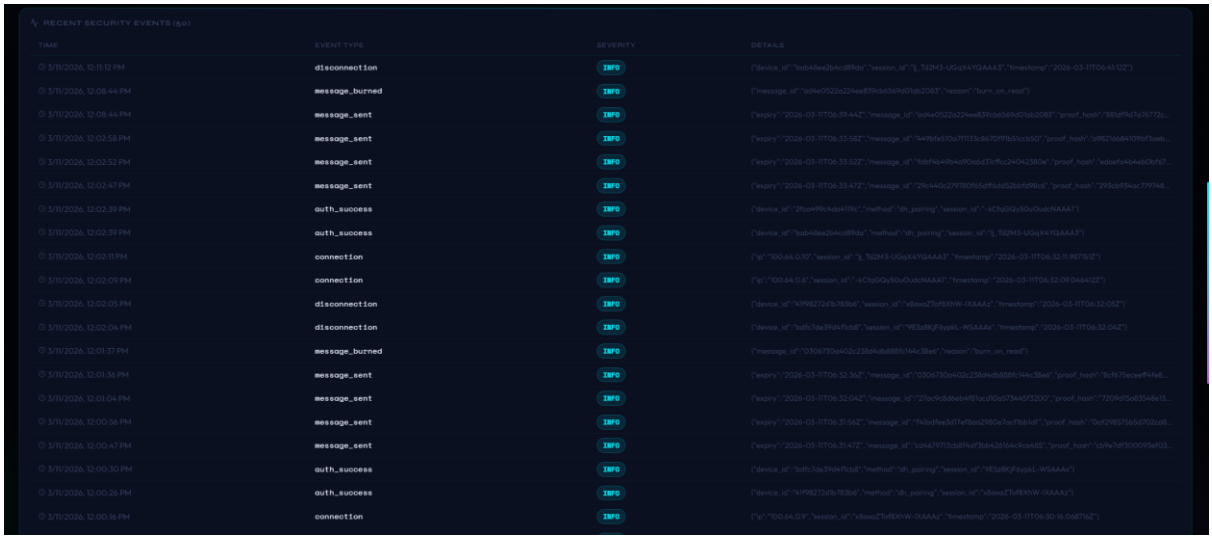


Explanation
The administrator's dashboard gives an overview of system security through the display of parameters like:

- Attacks detected
- Number of devices
- Number of proofs created
- Expiring messages

These are dynamic values which are continually updated depending on activities within the system.

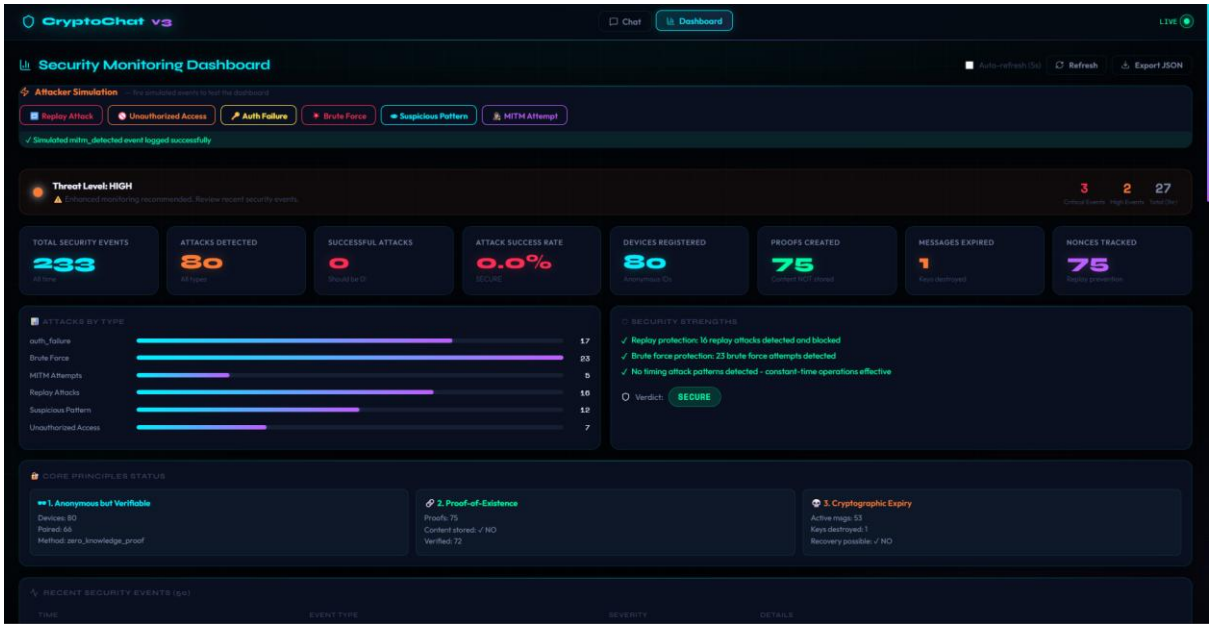
6.6 Activity Logs



Explanation
The logs contain records of all system activities including message exchange, authentication, connectivity, and deletion activities. The information logged includes timestamping, type of event, and severity of the event. The log system allows for:

- Traceability of system activity
- Anomaly detection
- Incident investigation

6.7 Attack Simulation



Explanation

The system contains a fully integrated attack simulator module to determine the system's ability to handle various forms of attacks including:

- Replay attacks
- Brute force attacks
- Man-in-the-middle attacks
- Unauthorised access attacks

As illustrated by the dashboard below, the system was able to detect and handle all the attacks without any problem.

7. Cryptanalysis

Cryptanalysis is an integral part of analyzing the strength of any cryptography algorithm, and involves the study of how robust the given algorithm is against certain types of attacks. While traditional systems depend on trust assumptions, the design of CryptoChat v2 allows the security provided by the algorithm to be cryptanalyzed and empirically tested against established cryptography attack methods.

7.1 Threat Model

7.1.1 Adversary Capabilities

The following are some of the capabilities of the attacker considered in our system:

- Ability to intercept all network communications (MITM capability)
- Ability to record encrypted messages for analysis at a later point
- Ability to perform replay attacks based on stored packets
- Ability to temporarily/harvest data from server - side storage
- Brute force attacks on encryption keys/parameters
- Identity/tracing attack

However, the computational power of the attacker is assumed to be constrained such that breaking AES-256 encryption, SHA-256 hashes, or Diffie-Hellman encryption is impossible within feasible time.

7.1.2 Security Objectives

Cryptanalysis checks if the system meets the following objectives:

- Message confidentiality
- Transmitted information integrity
- Forward secrecy
- User anonymity
- Verification of message existence
- Replay and impersonation attack resistance

7.2 Analysis of Core Cryptographic Algorithms

7.2.1 Diffie-Hellman Key Exchange Algorithm

Attacks Considered: Man-in-the-Middle (MITM):

- Unauthenticated DH is susceptible to MITM attacks.
- CryptoChat v2 prevents MITM attacks through Safety Numbers (fingerprints).

Analysis:

- MITM attacks are rendered impossible if the users validate their safety numbers.
- Vulnerability still exists otherwise, but it affects only the user, not the algorithm.

Conclusion:

Security depends on proper use; relies on user verification process.

7.2.2 Double Ratchet Algorithm

Attack Considered: Key Compromise Attack:

- An attacker cannot recover any of the previous keys by compromising a current key.

Evaluations:

- Provides forward secrecy
- Also offers post-compromise security

Conclusion:

- Very resilient against retrospective attacks.

7.2.3 AES-256-GCM Encryption

Attack Considered: Tampering and Forging of Ciphertext:

- The AES-GCM algorithm has authentication tags

Evaluation:

- Ciphertext modification leads to failed verification.
- Provides protection against tampering and forging.

Conclusion:

- Confidentiality and integrity ensured by encryption.

7.2.4 Merkle Tree Structure

Attack Considered: Tampering of Data:

- Modification of a single leaf changes the root hash

Evaluations:

- Tampering can be easily detected.
- Proves the immutability of data structure.

Conclusion:

- Implements an immutable proof of message history.

7.2.5 Ring Signatures

Attack Considered: Identifying Sender:

- An attacker tries to discover a sender from a signature ring.

Evaluations:

- Signatures provide evidence only of ring membership.
- In-computational for an attacker to find a signer.

Conclusion:

- Strong guarantee of anonymous signature.

7.2.6 Blind Signatures

Attack Considered: Token Traceability:

- Server links issued tokens with corresponding users

Evaluations:

- Blinding process prevents linking of signatures.
- Prevents any traceability of token issuances.

Conclusion:

- Offers full-proof anonymous authentication process.

7.2.7 Commitment Scheme
Attacks considered: Message Manipulation:
- The user attempts to alter message after commitment.

Analysis:
- Hash collision shows tampering; binding ensures no alteration.

Result:
- Provides non-repudiation and integrity.

7.2.8 Proof of Deletion
Attacks considered: False Deletion Claims:
- The server provides false evidence for deletion of keys.

Analysis:
- Proof based on HMAC proves deletion.
- It is verifiable by third parties.

Result:
- Avoids trust in deletion proofs.

7.2.9 Time-lock Encryption
Attacks considered: Delayed Message Decryption:
- The attacker retains the message ciphertext for later decryption.

Analysis:
- Encryption keys become invalid after expiration.
- Decryption mathematically becomes impossible.

Result:
- Enforces data expiration.

7.3 Attack Vectors
7.3.1 Replay Attack
- Nonce used for authentication.
- Repeated message discarded.

Outcome:
- Countermeasure successful.

7.3.2 Brute Force Attack
- Use of strong keys (256 bit long).
- Utilizing PBKDF2 algorithm with many iterations.

Outcome:
Unfeasible due to computational complexity.

7.3.3 MITM Attack
- Countermeasures using safety number.

Outcome:
Mitigation through user identification possible.

7.3.4 Unauthorized Access

- Connection allowed only between paired gadgets.
- Verification required before message exchange.

Outcome:
Access control successful.

7.3.5 Traffic Analysis

- Message content is always encrypted.
- Metadata leakage still possible.

Outcome:
Mitigation partial (acknowledged limitation).

7.4 Observations Regarding System-Level Security

7.4.1 Strengths

- No plain-text storage on server.
- End-to-end encryption is ensured.
- Proofs are used instead of trust.
- Layered security systems.
- Real-time security and threat detection.

7.4.2 Weaknesses

- Depends on the user for verification of safety numbers.
- Not completely free from metadata leaks.
- Ring size affects anonymity level.
- Not entirely quantum-proof cryptography.

7.5 Cryptographic Strength Summary

The system possesses robust resistance to the following attacks:

- Passive listening
- Active interference
- Identity tracking
- Key cracking attacks
- Data manipulation

The application of several cryptographic mechanisms results in the implementation of a defence-in-depth model that would not affect the overall system even upon cracking a single component.

8. Comparative Analysis/State-of-the-Art Analysis

The design of CryptoChat v2 is based on a number of cryptographic protocols that have reached the status of state-of-the-art in modern communication channels. The key difference from the traditional approach is that CryptoChat v2 uses several sophisticated methods to ensure guaranteed security.

8.1 Evaluation Criteria

To conduct a proper comparison, the following parameters are used:

- End-to-End Encryption
- Forward Secrecy
- Anonymity of Users
- Security of Metadata
- Verifiable Data
- Management of Keys
- Persistence and Deletion of Data
- Protection Against Attacks

These criteria form the basis of any modern communication platform.

8.2 Comparison with Other Systems

8.2.1 Signal Protocol

Signal is generally considered the most secure messaging platform available today.

Features:

- End-to-end encryption.
- Double Ratchet algorithm.
- Forward secrecy.
- Verification of safety numbers.

Comparison:

- CryptoChat v2 utilizes Double Ratchet mechanism just like Signal
- Both offer strong forward secrecy
- However, Signal lacks:
- Cryptographic proof of existence of a message
- Proof of message destruction
- Anonymity enabled by ring signatures

Conclusion:

CryptoChat v2 enhances the capabilities of the Signal protocol through verifiability and anonymity, which exceed confidentiality.

8.2.2 WhatsApp

WhatsApp leverages the Signal protocol while relying on centralized elements.

Features:

- End-to-end encryption.
- Metadata gathering (user identification, timestamps, contact list).
- Cloud backups (not necessarily encrypted).

Comparison:

- CryptoChat v2 does not require identity verification (no need for phone number).
- No plaintext or message storage on server.
- Less metadata exposure.
- Offers cryptographic proofs for each operation.

Conclusion:
CryptoChat v2 surpasses WhatsApp in terms of user privacy and data protection.

8.2.3 Telegram

Telegram supports regular and secret messages.

Features:

- Regular messages are not encrypted.
- Secret messages are encrypted but fail to use key rotation effectively.
- Low forward secrecy support.

Comparison:

- CryptoChat v2 ensures end-to-end encryption in all cases.
- Uses Double Ratchet algorithm for dynamic key updates.
- Delivers stronger cryptographic assurances.

Conclusion:
CryptoChat v2 offers superior security in comparison to Telegram's baseline.

8.2.4 Blockchain-based messaging protocols

Some contemporary protocols implement blockchain technology for message authentication purposes.

Features:

- Unalterable record
- Publicly verifiable information
- Large computational costs

Comparison:

- CryptoChat v2 employs Merkle Trees instead of implementing blockchain technology.
- Proof of existence with minimal computational cost.
- Greater efficiency compared to blockchain-based protocols.

Conclusion:
CryptoChat v2 offers blockchain-equivalent security features at a reduced cost.

8.3 Feature Comparison Table

Feature / Property	Signal	WhatsApp	Telegram	CryptoChat v2
End-to-End Encryption	Yes	Yes	Partial	Yes
Forward Secrecy	Yes	Yes	Limited	Yes
Anonymous Authentication	No	No	No	Yes
Ring Signatures	No	No	No	Yes
Blind Signatures	No	No	No	Yes
Merkle Proof of Existence	No	No	No	Yes
Proof of Deletion	No	No	No	Yes
Metadata Protection	Partial	Low	Medium	High
Key Expiry Enforcement	No	No	Limited	Yes
Verifiable Security Claims	No	No	No	Yes

8.4 Current State-of-the-Art Techniques Employed

The CryptoChat v2 protocol employs several highly advanced cryptographic techniques that can be regarded as state-of-the-art in secure systems:

8.4.1 Double Ratchet Algorithm

- Applied to Signal protocol.
- Guarantees forward secrecy and post-compromise security.

8.4.2 Merkle Trees

- Applied to Bitcoin.
- Efficient and verifiable data structure.

8.4.3 Ring Signatures

- Applied to privacy-based cryptocurrencies.
- Guarantees sender anonymity.

8.4.4 Blind Signatures

- Applied to anonymous digital cash systems.
- Guarantees unlinkability.

8.4.5 Commitments

- Applied to cryptographic protocols.
- Guarantees data integrity.

8.5 CryptoChat v2 Innovations

Whereas other methods implement various cryptographic methods individually, CryptoChat v2 presents the following innovations:

8.5.1 Secure Verification Model

As opposed to systems that depend on trust:

- All claims have cryptographic evidence attached.
- Users can verify their own actions within the system.

8.5.2 Proof of Data Deletion

- Proofs that data was deleted.

- Prevents any server deception.

8.5.3 Anonymous Communication Protocol

- Identity is not mandatory (no phone number/e-mail).
- Ring signatures provide untraceable communication.

8.5.4 Multi-Level Security Design

- Uses many cryptographic primitives together.
- Defensive-in-depth security design.

8.6 Constraints Relative to Best Practices

Even with these advantages, there are some constraints to CryptoChat v2:

- It does not include post-quantum algorithms
- Metadata leaks can occur
- Multiple encryption layers reduce performance
- Size of ring signatures may be a concern

9. Conclusion

Cryptochat v2 shows that a secure communication application does not require an element of trust in any entity or institution but relies on mathematical evidence of guarantees and security. Instead of asking users to believe the platform about their privacy and confidentiality based on policies, CryptoChat v2 uses cryptographic proof as a replacement for every statement and policy.

Nine cryptographic primitives are used by this system. They are Diffie-Hellman key exchange, which creates a secure connection between parties, Double Ratchet protocol providing forward secrecy, AES-256-GCM, which encrypts data ensuring its confidentiality and integrity, Merkle Trees, which prove that a certain message exists in the database without disclosing it, as well as ring and blind signatures ensuring anonymity of the senders.

In addition, commitment schemes, Proof of Deletion, and Time-Lock Encryption ensure that data lifecycle management is cryptographically ensured. The system withstands any type of attack by using nonce, PBKDF2, and safety numbers verification.

Limitations of this system include possible metadata leaks, safety number verification requirement, as well as lack of post-quantum cryptographic primitives.

To conclude, CryptoChat v2 perfectly represents the philosophy "From Trust Me to Prove It".

*****